PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
   int key;
   struct Node *left;
   struct Node *right;
};
struct Node* createNode(int key) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->key = key;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}
struct Node* insert(struct Node* root, int key) {
   if (root == NULL) {
      return createNode(key);
   }
   if (key < root->key) {
      root->left = insert(root->left, key);
   }
   else if (key > root->key) {
      root->right = insert(root->right, key);
   }
```

```
    return root;
}
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}
void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->key);
        preorder(root->left);
        preorder(root->right);
    }
}
void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->key);
    }
}

int bstSearch(struct Node* root, int key, int *comparisons) {
    while (root != NULL) {
```

```
        (*comparisons)++;
        if (key == root->key) {
            return 1;
        }
        else if (key < root->key) {
            root = root->left;
        }
        else {
            root = root->right;
        }
    }
    return 0;
}
int linearSearch(int arr[], int n, int key, int *comparisons) {
    for (int i = 0; i < n; i++) {
        (*comparisons)++;
        if (arr[i] == key) {
            return 1;
        }
    }
    return 0;
}
int main() {
    int isbn[] = {45, 20, 60, 10, 30, 50, 70, 25, 55};
    int n = sizeof(isbn) / sizeof(isbn[0]);
```

```
struct Node* root = NULL;
for (int i = 0; i < n; i++) {
   root = insert(root, isbn[i]);
}
printf("BST Traversals\n");
printf("----------------------------\n");
printf("Inorder   : ");
inorder(root);
printf("\n");
printf("Preorder  : ");
preorder(root);
printf("\n");
printf("Postorder : ");
postorder(root);
printf("\n\n");
int searchKeys[] = {25, 55, 90};
int numSearches = 3;
printf("Search Results\n");
printf("------------------------------------------------\n");
printf("Key\tBST Result\tBST Comparisons\tLinear Result\tLinear Comparisons\n");
printf("------------------------------------------------\n");
for (int i = 0; i < numSearches; i++) {
   int key = searchKeys[i];
   int bstComparisons = 0;
   int linearComparisons = 0;
```

```
        int bstResult = bstSearch(root, key, &bstComparisons);
        int linearResult = linearSearch(isbn, n, key, &linearComparisons);
        printf("%d\t%s\t\t%d\t\t%s\t\t%d\n",
            key,
            bstResult ? "Found" : "Not Found",
            bstComparisons,
            linearResult ? "Found" : "Not Found",
            linearComparisons);
    }
    return 0;
}
```

OUTPUT

Output

```
BST Traversals
----------------------------
Inorder   : 10 20 25 30 45 50 55 60 70
Preorder  : 45 20 10 30 25 60 50 55 70
Postorder : 10 25 30 20 55 50 70 60 45

Search Results
-------------------------------------------------
Key BST Result  BST Comparisons Linear Result   Linear Comparisons
-------------------------------------------------
25  Found       4       Found       8
55  Found       4       Found       9
90  Not Found       3       Not Found       9
```